Measuring Semantic Preservation in Passkey Migration

Anonymous Authors

Abstract

An imported passkey can still authenticate while a property needed by an application has changed. We present PasskeyTransit, a reproducible harness that measures identity, WebAuthn behavior, extension state, and operational guarantees separately. A balanced synthetic campaign exercised 6,144 credential–route–repetition attempts; 5,120 reached import and 1,024 were rejected before a ceremony. Every executed assertion authenticated, yet 512 imported attempts failed an executed browser behavior oracle for user verification or largeBlob. These fractions describe the constructed matrix, not product prevalence. Follow-up controls distinguish a project-authored CXP binding from selected provider behavior. A Chromium virtual-authenticator transfer preserved the signing credential but lost its PRF result; four public assertions from a replay were independently reverified. Fenko authenticated after CXF import while losing the original HMAC seeds. A Proton software authenticator preserved both seeds and reproduced the expected PRF with user verification. We provide claim boundaries and public verifiers. We did not establish public CXP wire interoperability between providers or a verified Android transfer.

1 Introduction

An accepted import and successful login are weak tests of passkey migration. A relying party can also depend on the credential identifier, WebAuthn extension output, user-verification behavior, and whether transfer retries leave duplicate state. The Credential Exchange Format (CXF) represents credentials [1]; the draft Credential Exchange Protocol (CXP) addresses protected exchange [2]. WebAuthn authenticates an assertion [4], but a valid signature does not by itself show that every credential property survived the move.

We model a credential as $C = (I, K, F, S)$: identity, signing material, observable functions, and operational guarantees. Our falsifiable proposition is that valid authentication does not imply preservation of every component. We contribute: (1) a claim matrix and property-specific oracles; (2) a synthetic campaign with explicit denominators and non-evaluable properties; (3) executable controls using two project-authored runtimes, Chromium virtual authenticators, and selected open-source provider code; and (4) an artifact whose public reports allow independent checks of retained signatures and PRF digest relationships. This is a measurement method with selected software cases, not a survey of deployed products.

2 Background and related work

CXF can encode FIDO2 HMAC state [1]. The CXP working draft defines a transport profile [2]; our single-document CXP/HPKE binding is an experimental project implementation and does not cover its full ZIP/JWE wire format. HPKE follows RFC 9180 [3]. A WebAuthn PRF result is returned through client extension results, outside the ordinary signed assertion bytes [4]. Consequently, signature and PRF oracles answer separate questions. Ramat et al. compare passkey user experiences across 111 websites [5]; Jannett et al. measure deployment and security on passkey-enabled websites [6]. Büttner and Gruschka compare the security and availability of device-bound and synced credentials [7]. Those units are websites or credential models. Our unit is an attempted migration of a controlled credential, with separate post-import oracles for signing, extension behavior, and retry state. We do not infer website adoption or provider prevalence from it.

3 Study design

3.1 Units, outcomes, and exclusions

The v1.5 protocol crossed 256 synthetic ES256 credentials (32 in each of eight strata) with 12 routes and two repetitions: 6,144 attempts in 96 route-by-stratum groups. The strata covered basic credentials, PRF with and without UV, largeBlob, credBlob, a payment-format marker, combined extensions, and a future optional member. They were constructed controls, not random draws. A route could reject the document

Table 1: C1 synthetic attempts; denominator 6,144.

Classification	Count	Share
Complete observable PASS	2,304	37.50%
Visible degradation	960	15.62%
Silent degradation	1,344	21.88%
Not evaluable	512	8.33%
Pre-ceremony rejection	1,024	16.67%

before WebAuthn. For each applicable property we record PASS, FAIL, NOT_APPLICABLE, or NOT_EVALUABLE. The last outcome is needed where CDP cannot inject PRF or credBlob state.

One oracle checks CXF representation and policy. Each imported attempt performs two WebAuthn assertions: the first binds a challenge to the attempt, credential, route, repetition, public key, user handle, RP ID, and origin; the second binds its challenge to the first and to a canonical digest of extension observations. A separate verifier checks both ES256 signatures, the RP hash, user-presence and verification flags, and counter. Extension oracles examine executed behavior where the interface permits it; representation loss alone is never relabeled as an executed PRF failure. A deterministic in-memory simulation tests transfer atomicity and retry behavior. These oracles are not interchangeable. Campaign percentages are design-weighted proportions, without population confidence intervals; paired repetitions are stability checks, not independent providers. The browser and provider follow-ups were designed after inspecting earlier results and are exploratory rather than preregistered confirmations.

3.2 Threat and implementation boundary

Two separately written project implementations exchange one CXF document in Python-to-Node and Node-to-Python directions and reject a modified ciphertext. This tests that experimental binding. Exporter identity, authorization, durable provider storage, and the public CXP ZIP/JWE envelope are outside the tested profile. The transaction model is in memory and makes no claim about crash durability.

4 Synthetic campaign results

Of 6,144 attempts, 1,024 were rejected before ceremony and 5,120 were imported. Every executed assertion among the imported attempts was accepted. Table 1 separates those accepted assertions from other oracles. The complete observable PASS yield was 2,304/6,144 (37.50%). This describes the balanced test design, not the probability that an arbitrary user or product will succeed.

Among imported attempts, 512 (10.00%) combined valid login with failure of an executed UV or largeBlob browser oracle. PRF or credBlob representation loss occurred in 1,792 (35.00%); this category overlaps the former and does not establish an executed PRF failure in those cells. The union of non-payment failures was 2,048/5,120 (40.00%). A payment-format marker raises the design sensitivity to 2,304/5,120 (45.00%), but is not a Secure Payment Confirmation ceremony. C2 malformed document controls comprised 80 cases across ten families and eight strata; 64 were rejected by structural validation, duplicate policy, or strict preservation. C3 exercised 960 in-memory failure sequences and 1,920 events: 832 rolled back and converged to one copy on retry, while the 128 deliberate legacy late-failure controls retained a provisional and duplicated on retry. Neither C2 nor C3 is included in the C1 percentages.

5 Executable follow-up controls

5.1 Experimental exchange and browser PRF loss

Python and Node recovered identical CXF documents in both directions and rejected ciphertext alteration. Software PRF evaluators agreed on retained CXF HMAC seeds and detected their removal. This is a functional software control, not a browser ceremony.

A Chromium virtual authenticator created a PRF-enabled passkey. Three source assertions showed a PRF result stable for one salt and different for a second. The runner exported the signing credential via CDP, carried the CXF document through the experimental exchange, and injected the credential into a new virtual authenticator. A fourth assertion authenticated at the destination, while the PRF result for the original salt was absent. CDP did not expose a PRF seed. This is a loss control for that test path, not a finding about a commercial credential provider.

A Windows 2025 GitHub Actions replay recorded six exchange and nine browser checks as true. A second replay retained four public signatures, the registration public key, challenges, authenticator and client data, and SHA-256 digests of observed PRF outputs. A separate verifier checked four ES256 signatures, RP binding, challenge uniqueness, UV flags, credential identity, and four PRF digest relationships. The archive contains no private key, seed, or raw PRF output. Comparing hashes proves relations among observed outputs, not the HMAC computation from a seed that CDP did not reveal.

5.2 Selected provider case: Fenko

We selected Fenko Vault 0.12.0 [8] after inspecting its CXF path. Two fresh profiles per repetition imported synthetic basic and PRF-bearing credentials. The credentials signed valid assertions. A public verifier rechecked 16 signatures and six PRF observation bindings. The source CXF HMAC

seeds were not retained in the observed import/export path; the browser PRF result differed from an oracle derived from those source seeds.

A same-provider round trip between two Fenko profiles produced matching PRF outputs in two selected repetitions. A separate verifier recalculated 24 signatures and 12 PRF bindings. This continuity does not imply that the original seeds survived: this code path derived a value from the imported signing key. Functional equality and preservation of the represented secret are separate claims. Two repetitions are not a product-level reliability estimate.

5.3 Positive software control: Proton

The Proton Pass common 2.0.0 software authenticator [9] provided a selected positive control. In two exploratory repetitions, the runner imported CXF, exported it, and reimported into a second software authenticator. Both HMAC seeds survived. With user verification, the expected PRF was reproduced for two salts before and after reimport. A modified-seed control changed the result while the credential still signed. The public verifier rechecked 12 signatures and 12 PRF digest comparisons, including four modified-seed controls. The isolated report cannot reconstruct the HMAC from a withheld seed; that requires repeating the pinned software run. This route used pinned WASM, not a browser passkey provider or the installed Proton Pass application; no no-UV ceremony was run.

6 Discussion and limitations

The browser loss control and Fenko case show different ways a signing credential can outlive its original PRF state. The Proton software route shows that preserving CXF seeds and reproducing the expected PRF is possible in one selected implementation. These cases are not a representative sample of provider behavior.

The public CXP wire format was not exchanged between two independent providers. A human-reported Android Bitwarden-to-Google transfer lacks an instrumented destination assertion and transfer receipt and is excluded from measured results. CDP cannot inject original PRF state; positive browser-to-browser restoration of CXF HMAC seeds is NOT_EVALUABLE. The Proton module result cannot fill that gap. The transaction simulation cannot support claims about storage durability, and the synthetic campaign cannot support product prevalence or superiority claims.

7 Conclusion

Passkey migration needs separate oracles for identity, signing, extension behavior, and transaction semantics. PasskeyTransit records when an interface makes a property unobservable. Its designed campaign and selected software controls demonstrate that valid authentication can coexist with a changed or missing property. The public replay lets readers recompute browser-control signatures without private credential material. Provider interoperability and verified mobile transfer require separate experiments.

A Open Science

The review package contains source, locked dependencies, protocols, synthetic reports, two GitHub Actions ZIP archives for the browser control, public signature transcripts, and verification scripts. `python tools/verify_v08_draft.py` checks pinned hashes and the phase-14, Fenko, and Proton public reports. A clean extraction with newly installed locked dependencies passed the same checks on 20 October 2026 on the same team's Windows host. This was an environment-isolation check, not independent replication. Provider vaults, private keys, original PRF seeds, and raw PRF outputs are excluded. Repeating provider paths requires pinned external revisions. An anonymous artifact URL without tracking must be provided before submission; the current public repository is identifying.

B Ethical considerations

All generated credentials and RP accounts in the reported software experiments were synthetic. The browser control used virtual authenticators and a localhost RP. No real user passkey or private vault is distributed. The excluded Android observation is not presented as independently verified evidence. We report provider behavior with route and revision scope rather than a general vendor security defect.

References

[1] FIDO Alliance. Credential Exchange Format v1.0 Proposed Standard Errata, 2026. https://fidoalliance.org/specs/cx/cxf-v1.0-ps-errata-20260309.html.

[2] FIDO Alliance. Credential Exchange Protocol v1.0 Working Draft, 3 October 2024. https://fidoalliance.org/specs/cx/cxp-v1.0-wd-20241003.html.

[3] R. Barnes et al. Hybrid Public Key Encryption. RFC 9180, 2022. https://www.rfc-editor.org/rfc/rfc9180.

[4] W3C. Web Authentication Level 3, Recommendation, 25 August 2026. https://www.w3.org/TR/2026/REC-webauthn-3-20260825/.

[5] B. Ramat, D. Kartchner, M. Clark, and K. Seamons. Passkeys in the Wild: A Systematic Study of FIDO2 User Experience Consistency Across Websites. SOUPS, pp. 21–39, 2026. https://www.usenix.org/conference/soups2026/presentation/ramat.

[6] L. Jannett, A. Mayer, M. Westers, V. Mladenov, C. Mainka, and J. Schwenk. The State of Passkeys: Studying the Adoption and Security of Passkeys on the Web. USENIX Security, pp. 5039–5058, 2026. https://www.usenix.org/conference/usenixsecurity26/presentation/jannett.

[7] A. Büttner and N. Gruschka. Device-Bound vs. Synced Credentials: A Comparative Evaluation of Passkey Authentication. ICISSP, 2025. https://arxiv.org/abs/2501.07380.

[8] FenkoHQ. Passkey Vault 0.12.0, pinned commit `2d873fbe25257250de6e322e9d785b97b17a544f`. https://github.com/FenkoHQ/passkey-vault.

[9] Proton. Proton Pass common 2.0.0, pinned commit `69a0927f0fe7071243d6c38e404459a69210fdd4`. https://github.com/protonpass/proton-pass-common.